Policy Authoring and Management

> **Series:** IAM | **Notebook:** 4 of 10 | **Created:** January 2026 | **Last Updated:** 02/27/2026

Mastering Dynatrace Policy Syntax

Policies are the heart of Dynatrace Gen3 IAM. They define what actions users can perform. This notebook provides a comprehensive guide to policy authoring, from basic syntax to advanced patterns.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	`account-iam-admin` to create/modify policies
Prior Knowledge	**IAM-01** through **IAM-03**

1. Policy Fundamentals

Policies define **what actions** users can perform. They're separate from boundaries (which control **what data** users can see).

Policy Scope Levels

Scope	Applies To	Managed By
Account	All environments in account	Account Admin
Environment	Single environment	Environment Admin

Policy Assignment

Policies are assigned to **groups**, not individual users:

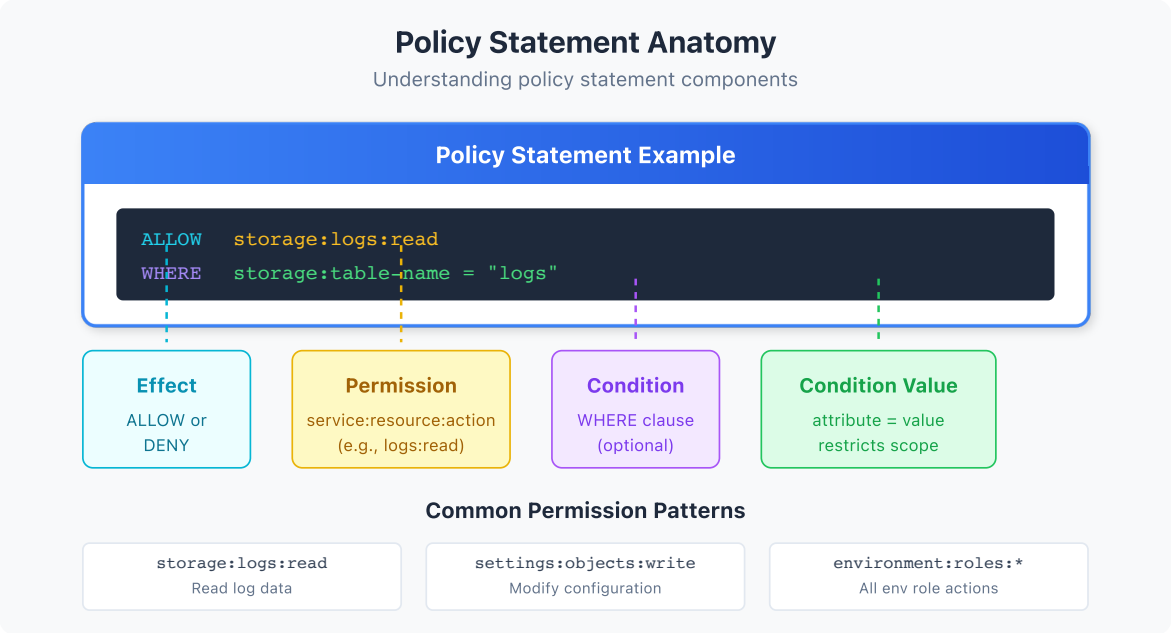
```
Group: dt-checkout-editors
├── Policy: environment-editor (default)
├── Policy: log-management (custom)
└── Boundary: checkout-services-only
```

Policy Types

Type	Description	Examples
Default Policies	Pre-built by Dynatrace	`environment-viewer`, `environment-editor`
Custom Policies	Created by you	`log-read-only`, `dashboard-manager`

2. Policy Statement Syntax

Every policy contains one or more **statements** that define allowed actions.



Basic Statement Structure

```
ALLOW &lt;service&gt;:&lt;resource&gt;:&lt;action&gt;
```

Examples:

Statement	Meaning
`ALLOW storage:logs:read`	Read all logs
`ALLOW storage:spans:read`	Read all spans
`ALLOW settings:objects:write`	Modify settings
`ALLOW document:documents:read`	Read dashboards/notebooks

Wildcards

Use `*` to match multiple values:

Statement	Matches
`ALLOW storage:*:read`	Read any storage type (logs, spans, metrics)
`ALLOW storage:logs:*`	Any action on logs (read, write)
`ALLOW *:*:read`	Read access to everything

Multiple Statements

Combine statements to build complete policies:

```
ALLOW storage:logs:read;
ALLOW storage:spans:read;
ALLOW storage:metrics:read;
ALLOW document:documents:read;
```

> **Note:** Statements are additive. Multiple ALLOW statements combine to grant broader access.

3. Services and Actions Reference

Core Services

Service	Purpose	Common Resources
`storage`	Grail data access	`logs`, `spans`, `metrics`, `events`, `bizevents`
`settings`	Configuration objects	`objects`, `schemas`
`document`	Notebooks, dashboards	`documents`, `directShares`
`environment`	Environment management	`extensions`, `activeGates`, `oneAgents`
`state`	Entity management	`entities`, `problems`
`automation`	Workflows, AutomationEngine	`workflows`, `calendars`

Common Actions

Action	Description
`read`	View/query data
`write`	Create/modify data
`delete`	Remove data
`share`	Share with others
`execute`	Run (for workflows)

Storage Service Details

Resource	Description	Actions
`logs`	Log records	read, write
`spans`	Distributed traces	read, write
`metrics`	Metric data	read, write
`events`	Platform events	read, write
`bizevents`	Business events	read, write
`buckets`	Storage buckets	read, write, delete

Settings Service Details

Resource	Description	Actions
`objects`	Configuration objects	read, write, delete
`schemas`	Schema definitions	read

Document Service Details

Resource	Description	Actions
`documents`	Dashboards, notebooks	read, write, delete, share
`directShares`	Direct share links	write

4. Condition Expressions

Conditions add fine-grained control to policy statements.

Condition Syntax

```
ALLOW &lt;service&gt;:&lt;resource&gt;:&lt;action&gt; WHERE &lt;condition&gt;
```

Condition Operators

Operator	Description	Example
`=`	Equals	`storage:dt.security_context = "team-a"`
`!=`	Not equals	`storage:dt.security_context != "restricted"`
`IN`	In list	`storage:dt.security_context IN ("team-a", "team-b")`
`startsWith`	Prefix match	`settings:schemaId startsWith "builtin:alerting"`
`contains`	Contains substring	`settings:schemaId contains "custom"`

Common Condition Fields

Domain	Field	Description
Storage	`storage:dt.security_context`	Security context on data
Storage	`storage:bucket.name`	**Bucket name (for team isolation)**
Settings	`settings:schemaId`	Settings schema
Settings	`settings:dt.security_context`	Security context on settings

Security Context vs Bucket Conditions

Condition Type	Use Case	Example
Security Context	Flexible, changeable access	`storage:dt.security_context = "team-a"`
Bucket	Hard data isolation	`storage:bucket.name = "team-a-logs"`
Both	Defense in depth	Both conditions in policy + boundary

Condition Examples

Scope to security context:

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout"
```

Scope to specific bucket (team isolation):

```
ALLOW storage:logs:read WHERE storage:bucket.name = "checkout_logs"
```

Scope to settings schema:

```
ALLOW settings:objects:write WHERE settings:schemaId startsWith
"builtin:alerting"
```

Multiple conditions (AND):

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "team-a" AND
storage:bucket.name = "team-a-logs"
```

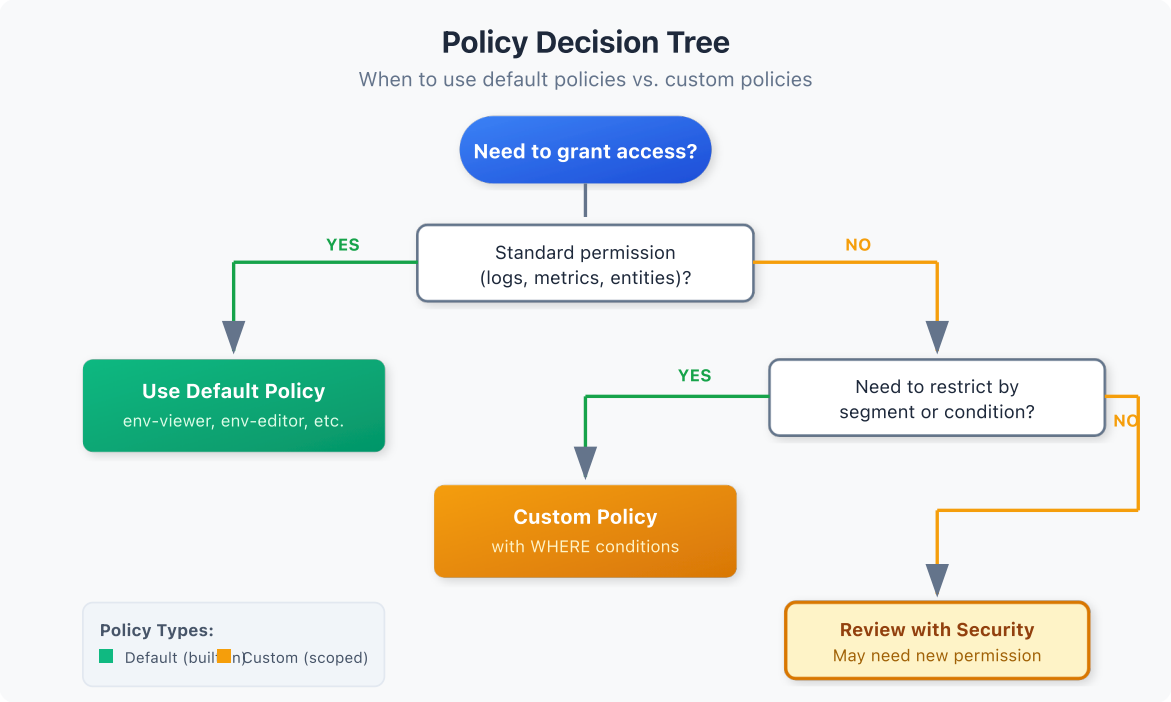
Multiple values (IN):

```
ALLOW storage:logs:read WHERE storage:dt.security_context IN ("team-a",
"team-b", "shared")
```

Multiple buckets (team access):

```
ALLOW storage:logs:read WHERE storage:bucket.name IN ("checkout_logs",
"shared_logs")
ALLOW storage:spans:read WHERE storage:bucket.name IN ("checkout_spans",
"shared_spans")
```

5. Common Policy Patterns



Pattern 1: Read-Only Access

```
// Read all data, no write access
ALLOW storage:*:read;
ALLOW document:documents:read;
ALLOW state:*:read;
ALLOW settings:objects:read;
```

Pattern 2: Team-Scoped Access (Security Context)

```
// Full access to team's data only (flexible, query-time filtering)
ALLOW storage:*:read WHERE storage:dt.security_context = "checkout-team";
ALLOW storage:*:write WHERE storage:dt.security_context = "checkout-team";
ALLOW settings:objects:* WHERE settings:dt.security_context = "checkout-
team";
```

Pattern 3: Bucket-Based Team Isolation (Recommended for Strict Isolation)

> **Use this pattern** when teams must have physically separated data with no cross-team visibility.

```
// Grant access to team's specific buckets (hard data isolation)
ALLOW storage:logs:read WHERE storage:bucket.name = "checkout_logs";
ALLOW storage:logs:write WHERE storage:bucket.name = "checkout_logs";
ALLOW storage:spans:read WHERE storage:bucket.name = "checkout_spans";
ALLOW storage:spans:write WHERE storage:bucket.name = "checkout_spans";
ALLOW storage:metrics:read WHERE storage:bucket.name = "checkout_metrics";
ALLOW storage:metrics:write WHERE storage:bucket.name = "checkout_metrics";

// Also grant access to shared buckets
ALLOW storage:logs:read WHERE storage:bucket.name = "shared_logs";
ALLOW storage:spans:read WHERE storage:bucket.name = "shared_spans";
```

Bucket + Security Context (Defense in Depth):

```
// Combine bucket restriction with security context for maximum isolation
ALLOW storage:logs:read WHERE storage:bucket.name = "checkout_logs"
  AND storage:dt.security_context = "checkout";
```

Pattern 4: Log Management

```
// Read all logs, manage log pipelines
ALLOW storage:logs:read;
ALLOW settings:objects:* WHERE settings:schemaId startsWith
"builtin:logmonitoring";
```

Pattern 5: Dashboard Creator

```
// Create and share dashboards
ALLOW document:documents:*;
ALLOW storage:*:read;
```

Pattern 6: Alerting Manager

```
// Manage alerting configuration
ALLOW settings:objects:* WHERE settings:schemaId startsWith
"builtin:alerting";
ALLOW settings:objects:* WHERE settings:schemaId startsWith
"builtin:problem";
ALLOW automation:workflows:*;
```

Pattern 7: SRE On-Call Access

```
// Broad read access + problem management
ALLOW storage:*:read;
ALLOW state:problems:*;
ALLOW document:documents:read;
ALLOW settings:objects:read;
```

Pattern 8: Security Auditor

```
// Read everything, write nothing
ALLOW storage:*:read;
ALLOW settings:*:read;
ALLOW document:*:read;
ALLOW state:*:read;
```

Pattern 9: Compliance-Scoped Access (Bucket-Based)

```
// Access only to PCI-compliant data bucket
ALLOW storage:logs:read WHERE storage:bucket.name = "pci_logs";
ALLOW storage:spans:read WHERE storage:bucket.name = "pci_spans";
// No access to non-PCI buckets
```

Choosing Between Patterns

Requirement	Recommended Pattern
Teams can share data flexibly	Pattern 2 (Security Context)
Teams must never see each other's data	**Pattern 3 (Buckets)**
Compliance/regulatory data separation	**Pattern 9 (Bucket-Based)**
Temporary project access	Pattern 2 (Security Context)
Cost allocation by team	**Pattern 3 (Buckets)**

6. Default vs Custom Policies

Built-in Default Policies

Dynatrace provides default policies for common use cases:

Policy	Description	Use Case
`environment-viewer`	Read-only access	Stakeholders, auditors
`environment-editor`	Read + write (most things)	Team members
`environment-admin`	Full environment control	Administrators
`account-viewer`	View account settings	Cross-team visibility
`account-iam-admin`	Manage IAM	IAM administrators

When to Use Default Policies

Scenario	Recommendation
Simple permission levels	Use defaults
Quick onboarding	Use defaults
Standard roles	Use defaults

When to Create Custom Policies

Scenario	Recommendation
Team-scoped data access	Custom policy
Specific capability (logs only)	Custom policy
Compliance requirements	Custom policy
Least privilege needs	Custom policy

Combining Default and Custom

Groups can have both:

```
Group: dt-checkout-editors
├── Default Policy: environment-viewer (base access)
└── Custom Policy: checkout-write-access (team-specific writes)
```

This provides base read access plus targeted write permissions.

7. Policy Testing and Validation

Testing Methodology

1. **Create policy in test environment** (or account)
2. **Assign to test group** with a test user
3. **Verify intended access** works
4. **Verify unintended access** is blocked
5. **Document results** and adjust

Common Validation Tests

Test	Expected Result
Query logs in scope	Success
Query logs out of scope	Empty or denied
Modify settings in scope	Success
Modify settings out of scope	Denied
Create dashboard	Success (if allowed)

Debugging Policy Issues

If access isn't working as expected:

1. **Check group membership** - Is user in the correct group?
2. **Check policy assignment** - Is policy assigned to group?
3. **Check boundary** - Does boundary allow access to the entity?
4. **Check conditions** - Do condition values match?
5. **Check security context** - Is data tagged correctly?

See **IAM-09: Troubleshooting Access Issues** for detailed debugging.

8. GitOps for Policies

Version control your policies for audit trail and peer review.

Policy as Code Structure

```
iam-config/
├── policies/
│   ├── team-checkout-access.yaml
│   ├── team-payments-access.yaml
│   ├── sre-oncall.yaml
│   └── security-auditor.yaml
├── groups/
│   └── group-assignments.yaml
└── README.md
```

Policy YAML Format (Monaco)

```
# policies/team-checkout-access.yaml
configs:
  - id: checkout-team-policy
    type:
      settings:
        schema: builtin:iam.policy
    config:
      name: checkout-team-access
      description: "Access policy for Checkout team"
      statements:
        - effect: ALLOW
          service: storage
          resource: "*"
          action: read
          condition: "storage:dt.security_context = 'checkout'"
        - effect: ALLOW
          service: storage
          resource: "*"
          action: write
          condition: "storage:dt.security_context = 'checkout'"
```

GitOps Workflow

1. **Branch**: Create feature branch for policy change
2. **Edit**: Modify policy YAML
3. **Review**: Submit PR, get peer review
4. **Test**: Deploy to test environment
5. **Approve**: Merge after validation
6. **Deploy**: CI/CD applies to production

Benefits

Benefit	Description
Audit Trail	Git history shows who changed what, when
Peer Review	Changes reviewed before apply
Rollback	Easy revert to previous state
Documentation	Policies are self-documenting
Consistency	Same process for all changes

See **AUTOM-03: Monaco** for full implementation details.

9. Policy Assessment Queries

Use these queries to audit policy-related activity.

```
// Track policy changes in audit log
fetch logs, from: now() - 30d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "policy")
| fields timestamp, content
| sort timestamp desc
| limit 50
```

```
// Find access denied events
fetch logs, from: now() - 7d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "denied") or matchesPhrase(content,
"unauthorized")
| fields timestamp, content
| sort timestamp desc
| limit 100
```

```
// Summarize permission-related audit events
fetch logs, from: now() - 7d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "permission") or matchesPhrase(content,
"policy")
| summarize eventCount = count(), by:{content}
| sort eventCount desc
| limit 20
```

Next Steps

With policies defined, complete your access control setup:

Recommended Path

1. **IAM-05: Boundary Design Patterns** - Control what entities users can see
2. **IAM-06: User Lifecycle and Provisioning** - Automate user management
3. **IAM-07: Audit Logging and Compliance** - Monitor policy effectiveness
4. **IAM-10: Templated Policy-Group Assignments** - Reusable parameterized policies for team/environment scoping at scale
5. **IAM-11: Policy Persona Workshop** - Design persona-based policies from stakeholder requirements using a structured 6-goal methodology

Policy Checklist

Before moving on, ensure you have:

- [ ] Understood policy statement syntax
- [ ] Reviewed services and actions reference
- [ ] Designed policies for your team structure
- [ ] Decided on default vs custom policy usage
- [ ] Established testing methodology
- [ ] Considered GitOps for policy management

Summary

In this notebook, you learned:

- Policy fundamentals and scope levels
- Statement syntax: `ALLOW <service>:<resource>:<action>`
- Services and actions reference
- Condition expressions for fine-grained control
- Seven common policy patterns
- When to use default vs custom policies
- Policy testing and validation approaches
- GitOps for policy version control

References

- Policy Statements Reference
- Permission Management
- Account Policies
